

LABORATORIO DE INGENIERÍA CON IA

Guía Maestra de Spec-Driven Development, Arnesees, Agentes y Guardrails

Manual integral de arquitectura moderna: desde la concepción de la especificación viva hasta el arnés automatizado de evaluación, la orquestación de agentes y los sistemas de contención.

AUTOR / ESTUDIANTE:

Christian Vargas A.

COPILOTO TÉCNICO:

Antigravity (Google DeepMind)

PROYECTO DE REFERENCIA:

SuperCart SDD (React 18 + Vite + Tailwind CSS)

FECHA Y VERSIÓN:

Octubre 2026 | Versión 1.0 Definitiva

1. ¿Qué es Spec-Driven Development (SDD)?

El **Desarrollo Guiado por Especificaciones (Spec-Driven Development o SDD)** es una metodología de ingeniería de software en la cual **una especificación formal, estructurada y versionada actúa como la Única Fuente de Verdad (Single Source of Truth o SSOT)** antes de escribir o modificar cualquier línea de código de producción.

El Problema de la Programación Tradicional con Agentes de IA

Cuando los desarrolladores utilizan IA pidiéndole **"hazme una pantalla de lista de compras"**, la IA recurre a sus patrones probabilísticos y **"adivina"** los nombres de los campos, inventa reglas de negocio, y genera código inconsistente. Esto produce el fenómeno de *alucinación técnica* y rompe el sistema ante el menor cambio.

La Regla de Oro del SDD

Ley Fundamental

"El código siempre sigue a la especificación; la especificación NUNCA persigue al código." Cualquier cambio de negocio (un campo nuevo, una regla de confirmación, un cambio de idioma) se aprueba primero en la especificación, luego se refleja en el arnés, y por último se programa.

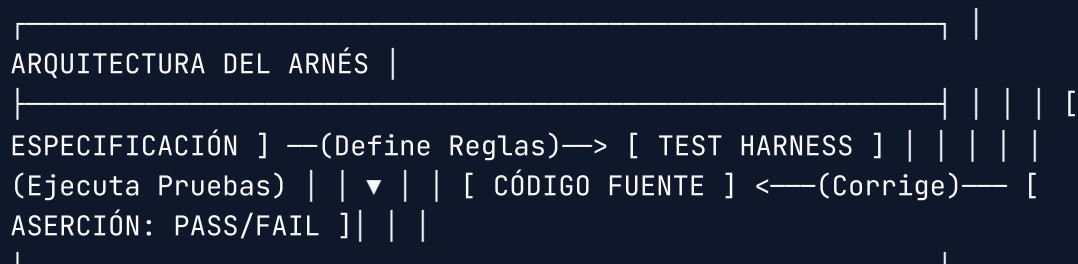
Estructura Anatómica de una Especificación Profesional

Un documento formal como `docs/ESPECIFICACION_PRINCIPAL.md` debe incluir obligatoriamente:

- **Historial de Cambios (Changelog):** Control de versiones SemVer (ej: v2.0 $\rightarrow$ v2.1 $\rightarrow$ v2.2) indicando fecha, autor y razón del cambio.
- **Modelo de Dominio / Contrato de Datos:** Definición exacta de entidades, claves, tipos y valores por defecto (TypeScript / JSON Schemas).
- **Requerimientos Funcionales (RF):** Reglas operativas numeradas y delimitadas (ej: **RF-1.1**, **RF-4.6**).
- **Requerimientos No Funcionales (RNF):** Políticas de persistencia, tolerancia a fallos, recuperación de corrupción y migración retroactiva.

2. El Arnés de Pruebas (The Test Harness)

El **Arnés de Pruebas** es el banco de evaluación automatizado donde se colocan los componentes de software en condiciones controladas para verificar que cumplen matemáticamente las promesas hechas en la especificación.



El Órgano Sensorial del Agente de IA

Los seres humanos verificamos el código abriendo el navegador, haciendo clics e inspeccionando la pantalla. **Un Agente de IA no tiene ojos humanos nativos.** El arnés es el *órgano sensorial* que le permite al agente saber si su solución funciona:

- **Sin Arnés:** El agente dice `"¡Listo, ya terminé!"` basándose en una corazonada. El código puede estar lleno de errores sutiles.
- **Con Arnés:** El agente ejecuta el arnés, lee el error exacto (ej: `"Expected 5 but received 2"`), corrige la línea de código precisa y vuelve a evaluar hasta obtener 100% verde.

La Metáfora del Restaurante de Alta Cocina

La Especificación (`SPEC.md`)

Es la **Carta y la Receta**. Define exactamente los ingredientes, el sabor, las porciones y lo que el cliente final pagó por recibir.

El Reglamento (`AGENTS.md`)

Son las **Normas de la Cocina**. Indica a los cocineros cómo comportarse, qué herramientas usar y qué prácticas están prohibidas.

El Arnés (`harness.html`)

Es el **Inspector de Calidad**. Antes de que el plato salga a la mesa, verifica peso, temperatura e higiene sin compasión.

La Memoria (`MEMORY.md`)

Es la **Bitácora del Chef**. Anota lecciones, incidentes pasados y recetas mejoradas para no repetir errores.

Principios de Aislamiento: El Sandbox (MockStorage)

Un buen arnés jamás toca los datos reales del usuario. Utiliza un **MockStorage** en memoria volátil. Cada prueba arranca con una pizarra en blanco, ejecuta su lógica y se limpia, garantizando pruebas reproducibles e independientes de la red o del disco.



3. El Ciclo de Vida: Rojo $\rightarrow$ Verde $\rightarrow$ Refactor

Heredado de Test-Driven Development (TDD) y perfeccionado para la era de la IA, este ciclo es la base de la máxima confiabilidad en ingeniería:

FASE	COLOR	ACCIÓN EN EL LABORATORIO	PROPÓSITO / OBJETIVO
1. Fase Roja	FAIL	Diseñar la prueba antes de programar y verla fallar con un error explícito.	Demostrar que la prueba es real y detecta la ausencia de la función.
2. Fase Verde	PASS	Escribir el código mínimo necesario para que la prueba pase a verde.	Alcanzar el cumplimiento estricto del contrato sin sobre-ingeniería.
3. Refactorización	CLEAN	Reorganizar taxonomía, limpiar deuda técnica, optimizar variables.	Mejorar la calidad sin miedo: el arnés garantiza que nada se rompió.

¿Por qué es indispensable ver la prueba en Rojo?



La Trampa de las Pruebas Falsas Positivas

Si creas una prueba y pasa en verde de inmediato sin que hayas programado nada, **la prueba no sirve** (está mal formulada o no está evaluando nada). La Fase Roja es la garantía científica de que la prueba es sensible a los fallos.



4. Arquitectura de Agentes y Subagentes

Al trabajar en Pair Programming con IA o con enjambres de múltiples agentes, cada actor debe tener un rol y límites perfectamente delimitados:

- **Agente Orquestador:** Entiende los objetivos de alto nivel del usuario, coordina la estrategia y gestiona la bitácora en `MEMORY.md`.
- **Subagente Programador (Coder):** Lee la especificación, revisa el archivo a modificar y escribe el código correspondiente.
- **Subagente Verificador (Tester / QA):** Corre el arnés de pruebas de manera independiente y certifica si el trabajo del programador pasa o debe rechazarse.

La Taxonomía Estandarizada de Pruebas

Una de las lecciones clave aprendidas hoy es que el arnés debe hablar el mismo idioma que la especificación. En lugar de identificadores genéricos (`test1` , `mod1`), cada prueba lleva el identificador contractual:

- `[RNF-02]` : Prueba de Resistencia ante JSON corrupto.
- `[RNF-03]` : Prueba de Migración Automática de versiones.
- `[RF-1.1]` : Prueba de Alta y Validación de Productos.
- `[RF-4.6]` : Prueba de Prevención de Desmarcado Accidental.

5. Guardrails: Las Barandillas de Seguridad

Los **Guardrails (Barandillas de Seguridad)** son restricciones, validaciones y cortafuegos diseñados para evitar que los agentes de IA o los propios usuarios cometan errores destructivos, rompan la arquitectura o provoquen pérdidas de datos.

1. Guardrails de Instrucción (AGENTS.md)

Límites operativos que la IA no puede violar: *"Prohibido asumir requerimientos", "Prohibido entregar tareas si el arnés está en rojo", "Obligatorio consultar docs/ antes de codificar".*

2. Guardrails Estructurales (El Arnés y CI/CD)

Sistemas automatizados que impiden la integración de código roto: Git hooks (Husky), pruebas unitarias previas al commit y pipelines de despliegue.

3. Guardrails de Negocio / UX (Anti-dedazos)

Validaciones en la experiencia de usuario. Ejemplo real implementado hoy (**RF-4.6**): si un producto ya está en el carrito, se exige confirmación obligatoria antes de desmarcarlo.

4. Guardrails de Resiliencia (Fallback)

Tolerancia ante desastres. Si la base de datos local se corrompe (**RNF-02**), el sistema captura la excepción y reinicia las tablas limpias sin colapsar la aplicación.

Implementación de Guardrails en Proyectos Reales

```
// Ejemplo de Guardrail de Código: Validación y Confirmación (RF-4.6)
alternarConConfirmacion(itemId, callbackConfirmacion) {
  const item = db.productos_listas.find(i => i.id === itemId);
  if (!item) return false;

  // Paso libre sin fricción hacia adelante
  if (!item.enCarrito) {
    item.enCarrito = true;
    return true;
  }

  // GUARDRAIL: Freno de seguridad antes de una acción potencialmente accidental
  const confirmado = typeof callbackConfirmacion === 'function' ? callbackConfirmacion() : true;
  if (confirmado) {
    item.enCarrito = false;
    return false;
  }

  // Si no se confirma, se mantiene protegido
  return true;
}
```

6. Checklist de Buenas Prácticas (Cheat Sheet)

Protocolo Profesional para Desarrollar Nuevas Funcionalidades

1. **Definir y Acordar:** Hablar con el usuario/sponsor para entender el problema real.
2. **Documentar en la Spec:** Crear o editar la sección correspondiente en `docs/ESPECIFICACION_PRINCIPAL.md` y aumentar la versión.
3. **Diseñar el Caso de Prueba:** Agregar la prueba en `tests/harness.html` con su código estandarizado (`RF-*`).
4. **Comprobar el Fallo Rojo:** Re-ejecutar el arnés y certificar que la nueva prueba falla con la causa esperada.
5. **Escribir el Código:** Modificar los servicios y componentes en `src/` para resolver la aserción.
6. **Comprobar el Éxito Verde:** Re-ejecutar el arnés hasta alcanzar 100% PASS.
7. **Registrar en la Memoria:** Agregar la decisión arquitectónica (ADR) en `MEMORY.md` .
8. **Commit Atómico en Git:** Guardar el cambio con un mensaje semántico claro (`feat(...)`).

SuperCart SDD – Laboratorio de Aprendizaje

Documento generado para Chris Vargas A. en colaboración con AntigraVity (Google DeepMind).